

# The Infernet Protocol Book

The complete guide to decentralized GPU inference

Infernet Protocol

2026

## Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>The Infernet Protocol Book</b>             | <b>3</b>  |
| Who This Book Is For . . . . .                | 3         |
| What You'll Learn . . . . .                   | 3         |
| Quick Reference . . . . .                     | 4         |
| Running the Examples . . . . .                | 4         |
| Contributing . . . . .                        | 4         |
| <b>Chapter 1: Introduction</b>                | <b>4</b>  |
| In This Chapter . . . . .                     | 4         |
| The Core Idea in One Paragraph . . . . .      | 5         |
| <b>Architecture</b>                           | <b>5</b>  |
| Component Overview . . . . .                  | 5         |
| Control Plane . . . . .                       | 6         |
| Node Daemon . . . . .                         | 6         |
| Inference Backends . . . . .                  | 6         |
| Job Flow . . . . .                            | 7         |
| Key Design Decisions . . . . .                | 7         |
| <b>Quick Start</b>                            | <b>7</b>  |
| Prerequisites . . . . .                       | 7         |
| Step 1: Install the CLI . . . . .             | 8         |
| Step 2: Get Your Registration Token . . . . . | 8         |
| Step 3: Run Setup . . . . .                   | 8         |
| Step 4: Start the Daemon . . . . .            | 9         |
| Step 5: Verify . . . . .                      | 9         |
| Step 6: Send a Test Job . . . . .             | 10        |
| What's Next . . . . .                         | 10        |
| <b>What Is Infernet Protocol?</b>             | <b>10</b> |
| The Problem . . . . .                         | 10        |
| What It Is . . . . .                          | 10        |
| How It Works at a High Level . . . . .        | 11        |
| What Makes It Different . . . . .             | 11        |

|                                           |           |
|-------------------------------------------|-----------|
| What It Is Not . . . . .                  | 11        |
| <b>Chapter 2: Node Operators</b>          | <b>12</b> |
| In This Chapter . . . . .                 | 12        |
| The Operator's Day-to-Day . . . . .       | 12        |
| <b>Earnings</b>                           | <b>12</b> |
| How Payments Work . . . . .               | 12        |
| Checking Your Balance . . . . .           | 13        |
| Setting Your Payout Address . . . . .     | 13        |
| Claiming Earnings . . . . .               | 14        |
| Payout Minimums . . . . .                 | 14        |
| Pricing Per Job . . . . .                 | 14        |
| Automated Payouts . . . . .               | 15        |
| Payment Security . . . . .                | 15        |
| <b>Installation</b>                       | <b>15</b> |
| Install the CLI . . . . .                 | 15        |
| Run Setup . . . . .                       | 16        |
| Firewall Configuration . . . . .          | 17        |
| Running as a System Service . . . . .     | 18        |
| Upgrading the CLI . . . . .               | 19        |
| Uninstalling . . . . .                    | 19        |
| <b>Model Management</b>                   | <b>19</b> |
| How Served Models Work . . . . .          | 20        |
| Installing Models via CLI . . . . .       | 20        |
| Removing Models via CLI . . . . .         | 20        |
| Listing Installed Models . . . . .        | 20        |
| Installing Models via Dashboard . . . . . | 21        |
| Model Naming Conventions . . . . .        | 21        |
| VRAM Planning . . . . .                   | 22        |
| Hot-Swap vs Cold-Load . . . . .           | 22        |
| <b>Monitoring Your Node</b>               | <b>22</b> |
| Status Overview . . . . .                 | 22        |
| Logs . . . . .                            | 23        |
| Doctor . . . . .                          | 24        |
| Dashboard Monitoring . . . . .            | 24        |
| Heartbeat Intervals . . . . .             | 25        |
| Alerting . . . . .                        | 25        |
| <b>Hardware Requirements</b>              | <b>25</b> |
| GPU Tiers . . . . .                       | 25        |
| RAM . . . . .                             | 26        |
| Storage . . . . .                         | 27        |
| Bandwidth . . . . .                       | 27        |
| Operating System . . . . .                | 27        |
| GPU Drivers . . . . .                     | 28        |

|                                      |           |
|--------------------------------------|-----------|
| <b>Chapter 3: Inference Backends</b> | <b>28</b> |
| In This Chapter . . . . .            | 28        |
| How Auto-Selection Works . . . . .   | 29        |
| Backend Adapter Interface . . . . .  | 29        |

## The Infernet Protocol Book

Infernet Protocol is a decentralized GPU compute network. Node operators register their GPU servers and get paid to run LLM inference. Developers submit jobs through a unified API and get responses back without depending on any single provider.

This book covers everything you need to know to run a node, build applications on the network, or understand how the protocol works under the hood.

---

### Who This Book Is For

**Node operators** who want to earn crypto by contributing GPU compute. You have an NVIDIA, AMD, or Apple Silicon machine and want to put it to work. Start with Chapter 2: Node Operators.

**Application developers** who want to call LLM inference without locking into OpenAI, Anthropic, or any other centralized provider. You want reliable APIs, streaming responses, and predictable costs. Start with Chapter 4: Building Apps.

**Protocol contributors** interested in the cryptographic architecture, payment flows, and key hierarchy. Start with Chapter 5: Protocol.

If you're new to Infernet entirely, read Chapter 1: Introduction first.

---

### What You'll Learn

**Chapter 1 — Introduction** What Infernet Protocol is, the problem it solves, and a high-level architecture tour. Includes a 5-minute quickstart so you can see the system working before diving into details.

**Chapter 2 — Node Operators** Hardware requirements, the full installation walkthrough, model management, monitoring your node, and how earnings and payouts work.

**Chapter 3 — Inference Backends** Ollama, vLLM, SGLang, Modular MAX, and llama.cpp. How each one works, when to use it, and how Infernet auto-selects between them.

**Chapter 4 — Building Apps** The REST API, streaming chat with SSE, job lifecycle management, and error handling. JavaScript and Python examples throughout.

**Chapter 5 — Protocol Internals** The Nostr-style secp256k1 auth system, Compute Payment Receipts, multi-chain wallet support, and the IPIP-0028 model key hierarchy.

**Chapter 6 — Advanced Topics** Multi-GPU setups with vLLM and Ray, self-hosting the control plane, and the distributed training roadmap.

---

## Quick Reference

---

| Task                       | Where to look                      |
|----------------------------|------------------------------------|
| Install a node             | 02-node-operators/installation.md  |
| Pick an inference backend  | 03-inference-backends/choosing.md  |
| Stream tokens from the API | 04-building-apps/streaming-chat.md |
| Understand auth headers    | 05-protocol/security.md            |
| Run a 70B model on 2 GPUs  | 06-advanced/multi-gpu.md           |

---

---

## Running the Examples

Most code examples in this book assume:

- `INFERNET_NODE_URL` points to your node (e.g. `http://localhost:3000`)
- `INFERNET_BEARER_TOKEN` is set with a valid bearer token from your control plane

```
export INFERNET_NODE_URL=http://localhost:3000
export INFERNET_BEARER_TOKEN=your_token_here
```

For the CLI examples, `inferneter` must be installed and on your `PATH`. See installation if it isn't.

---

## Contributing

This book is open source and lives in `docs/book/` in the main Infernet Protocol repository. Pull requests are welcome. Corrections, new examples, and translations are especially appreciated.

---

## Chapter 1: Introduction

This chapter explains what Infernet Protocol is, why it exists, and how the pieces fit together. By the end you'll have a mental model of the whole system and a working node (or API call) to prove it.

---

### In This Chapter

- What Is Infernet Protocol? — The problem with centralized AI and how a decentralized compute network solves it.
  - Architecture — Control plane, node daemon, inference engines, and the job flow from request to payment.
  - Quick Start — Install the CLI, run setup, and verify your node is online in under 5 minutes.
-

## The Core Idea in One Paragraph

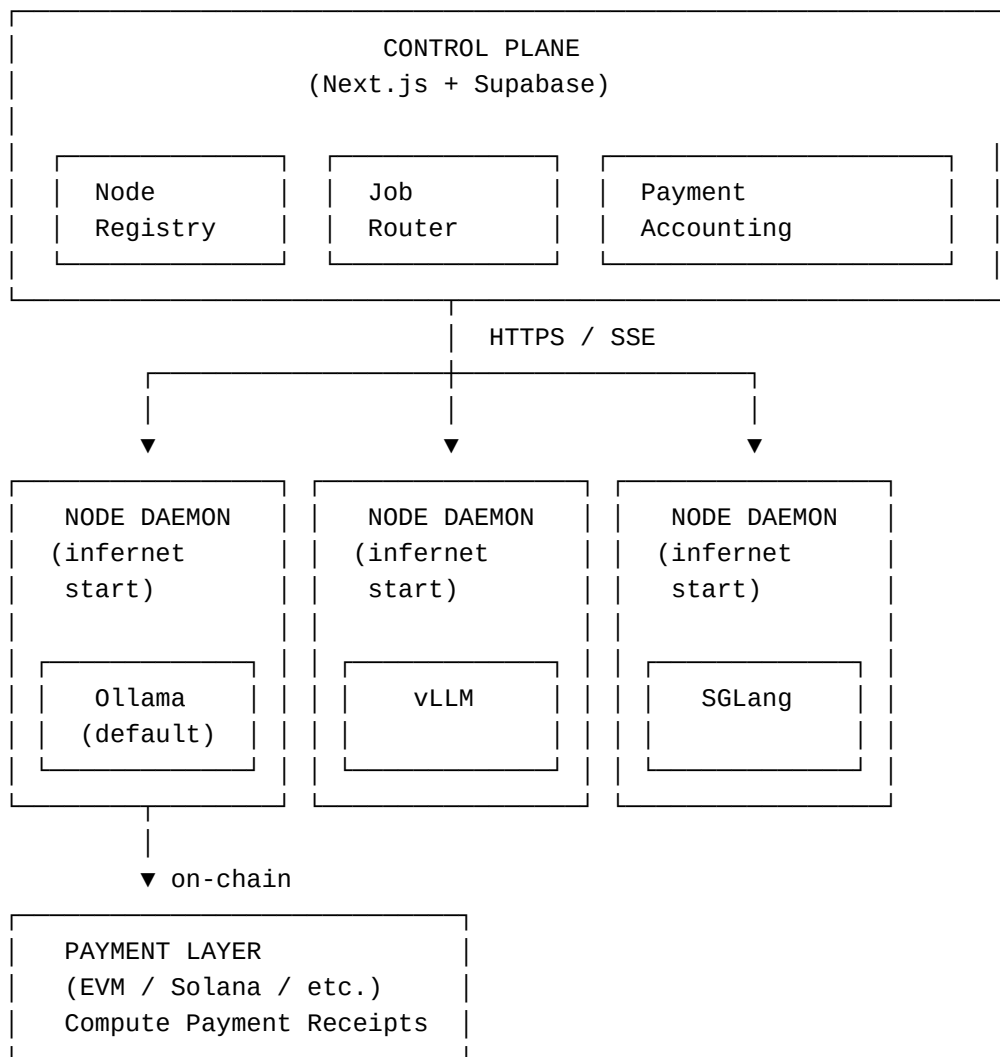
Anyone with a GPU can run a node. Anyone who needs LLM inference submits a job. The network routes the job to an available node with the right model loaded, the node runs inference and streams back the result, and the client's payment is settled on-chain. No single company controls routing, pricing, or what models are available. The cryptographic auth layer means nodes never need to trust the control plane with private keys.

---

## Architecture

Infernet Protocol has four major components: the control plane, node daemons, inference backends, and the on-chain payment layer. Here's how they connect.

### Component Overview



## Control Plane

The control plane is a Next.js application backed by Supabase. It serves two audiences:

**The dashboard** — a web UI for operators to see their nodes' status, earnings, model inventory, and recent jobs. Clients can also use the dashboard to browse available models and manage API access.

**The API** — the REST endpoints that clients call to submit jobs and that node daemons call to register, heartbeat, and poll for commands. The primary job submission endpoint is `POST /api/v1/jobs`.

Supabase handles persistence: node registrations, job records, payment accounting, and the command queue (model installs/removes). Supabase Realtime is used for live dashboard updates.

The control plane is open source. You can self-host it — see Chapter 6: Self-Hosting.

## Node Daemon

The node daemon is the process started by `infernet start`. It does several things:

**Heartbeat loop** — every 30 seconds, the daemon sends a signed heartbeat to the control plane. The heartbeat includes: node public key, IP address, port, GPU stats, which models are currently loaded, and current load. If the control plane doesn't hear from a node for 90 seconds, it marks the node offline.

**Command polling** — on each heartbeat cycle, the daemon checks a command queue in the control plane. Commands include `model_install` (pull a new model) and `model_remove` (evict a model). Operators issue these commands from the dashboard or CLI; the daemon picks them up and executes them.

**Job execution** — when a job is routed to the node, the daemon receives it, calls the local inference backend, and streams the result back.

**Auth** — every request the daemon makes is signed with the node's `secp256k1` private key. The signature covers the HTTP method, path, body hash, a nonce, and a timestamp. This is carried in the `X-Infernet-Auth` header. The control plane verifies the signature against the node's registered public key. The private key never leaves the node.

## Inference Backends

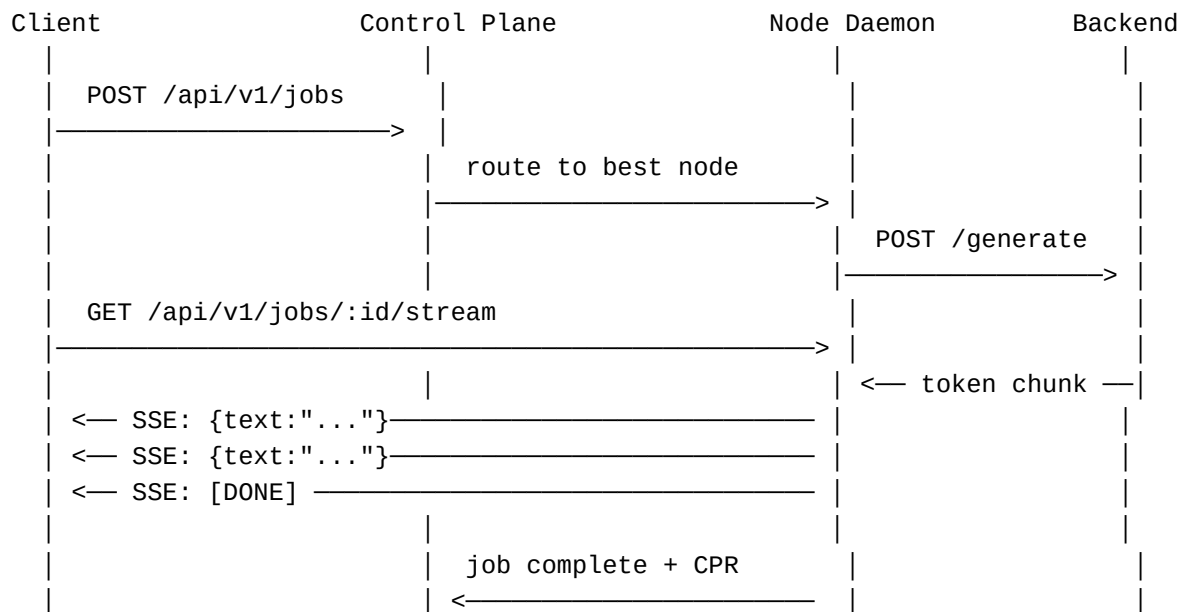
The daemon doesn't run inference itself. It delegates to one of five supported backends:

| Backend     | Best for                          | Protocol                             |
|-------------|-----------------------------------|--------------------------------------|
| Ollama      | General use, easy setup           | REST at <code>localhost:11434</code> |
| vLLM        | High-throughput NVIDIA            | OpenAI-compatible REST               |
| SGLang      | KV-cache reuse, structured output | OpenAI-compatible REST               |
| Modular MAX | Throughput, modern NVIDIA         | REST at configurable port            |
| llama.cpp   | CPU, Apple Silicon, GGUF models   | REST via llama-swap                  |

The daemon probes each backend in priority order at startup and uses the first one that responds. You can override the selection with env vars.

## Job Flow

Here's what happens from the moment a client submits a job to the moment they receive the last token:



The client can poll `GET /api/v1/jobs/:id` for status or open an SSE stream for real-time token delivery. Most applications use the stream.

## Key Design Decisions

**Supabase as the coordination layer** — not a custom blockchain. Job routing, node registry, and command queuing run on Postgres with real-time capabilities. This keeps latency low and the stack familiar. On-chain components handle only what needs to be on-chain: payment settlement.

**Nodes never trust the control plane with keys** — the `secp256k1` auth model means the control plane can verify node identity without ever holding private keys. A compromised control plane cannot impersonate a node or steal earnings.

**Backends are swappable** — the daemon speaks to all backends through a thin adapter layer. Adding a new backend is a matter of implementing the adapter, not changing the protocol.

**SSE for streaming** — Server-Sent Events are simpler than WebSockets for unidirectional streaming and work through most proxies and CDNs without special configuration.

---

## Quick Start

Get a node online in under 5 minutes. This walkthrough installs the CLI, runs setup, and verifies the node is heartbeating to the control plane.

### Prerequisites

- Linux (Ubuntu 22.04+ recommended) or macOS

- A GPU (or CPU-only for testing)
- `curl` available on PATH
- An account on the Infernet dashboard to get your registration token

If you're on a GPU machine, make sure your GPU drivers are installed before starting. The setup wizard will detect the GPU, but it can't install drivers for you.

## Step 1: Install the CLI

```
curl -sSL https://infernet.sh/install | bash
```

This script installs the `infernet` binary to `~/.local/bin` (or `/usr/local/bin` if you have write access) and adds it to your PATH. It takes about 30 seconds.

Verify the install:

```
infernet --version
# infernet 0.9.2
```

## Step 2: Get Your Registration Token

Open the Infernet Dashboard, sign in, and navigate to **Nodes** → **Add Node**. Copy the one-time registration token shown on that page. It looks like:

```
inft_reg_7x9k2mNpQwRsLvTbY4cJ
```

## Step 3: Run Setup

```
infernet setup
```

The setup wizard will:

1. Ask for your registration token
2. Detect your GPU (NVIDIA, AMD, Apple Silicon, or CPU)
3. Determine your VRAM tier (`>=48gb`, `>=24gb`, `>=8gb`, or `cpu`)
4. Install Ollama if no inference backend is detected
5. Ask which default model to load (defaults to `qwen2.5:7b` for `>=8gb`, `qwen2.5:1.5b` for `cpu`)
6. Generate a `secp256k1` keypair for this node
7. Register the node with the control plane
8. Write the config to `~/.infernet/config.json`

Example output:

```
Infernet Node Setup
=====
Registration token: inft_reg_7x9k2mNpQwRsLvTbY4cJ

Detecting hardware...
GPU: NVIDIA RTX 4090 (24 GB VRAM)
Tier: >=24gb
RAM: 64 GB
```



Checking inference backends...

Ollama: not found

Installing Ollama... done

Default model [qwen2.5:14b]:

Pulling qwen2.5:14b...  100% (8.1 GB)

Generating node keypair...

Public key: npub1abc123...

Config written to ~/.infernnet/config.json

Registering node...

Node ID: node\_8f3a2c1d

Status: registered

Setup complete. Run `infernnet start` to bring your node online.

## Step 4: Start the Daemon

```
infernnet start
```

The daemon starts in the foreground by default. You'll see heartbeat logs:

```
[2026-04-30 14:23:01] Node node_8f3a2c1d starting...  
[2026-04-30 14:23:01] Inference backend: ollama (localhost:11434)  
[2026-04-30 14:23:01] Loaded models: qwen2.5:14b  
[2026-04-30 14:23:02] Heartbeat OK (latency: 42ms)  
[2026-04-30 14:23:32] Heartbeat OK (latency: 39ms)
```

To run it in the background:

```
infernnet start --detach
```

Or install it as a system service that starts on boot:

```
infernnet service install
```

## Step 5: Verify

Check the node is online from the CLI:

```
infernnet status
```

```
Node:      node_8f3a2c1d  
Status:    online  
Uptime:    3 minutes  
Backend:   ollama  
Models:    qwen2.5:14b  
Jobs:      0 completed, 0 pending  
Earnings:  0.00 USDC
```

You should also see the node appear as **Online** in the dashboard within 30 seconds of starting the daemon.

## Step 6: Send a Test Job

You can send a test inference job directly from the CLI:

```
infernet chat "What is the capital of France?"
```

Paris.

Or with streaming visible:

```
infernet chat --stream "Explain how neural networks learn."
```

Tokens will appear as they're generated.

---

## What's Next

- **Node operators:** Read Chapter 2 for firewall setup, model management, monitoring, and earnings.
- **Developers:** Read Chapter 4 for the REST API, SSE streaming, and error handling.
- **Want a different backend?** Read Chapter 3 to swap Ollama for vLLM, SGLang, or MAX.

---

## What Is Infernet Protocol?

### The Problem

Modern AI applications depend almost entirely on a handful of centralized API providers. When OpenAI has an outage, thousands of products break. When Anthropic changes its pricing, startups scramble. When a model gets deprecated, months of prompt engineering disappears overnight.

Beyond reliability and cost, there's a structural problem: the compute that runs these models is controlled by a small number of large cloud providers. GPUs are expensive. If you have one, there's currently no straightforward way to monetize it by running inference for others. If you need inference, you're stuck paying whatever the incumbent providers charge.

Infernet Protocol is built to fix both sides of this.

### What It Is

Infernet Protocol is a decentralized GPU compute network for LLM inference. It has two kinds of participants:

**Node operators** run GPU servers with inference software installed. They register their nodes on the network, keep models loaded, and accept inference jobs. They earn crypto payments per job completed.

**Clients** submit inference jobs through a unified API. Jobs are routed to available nodes, executed, and results are streamed back. Clients pay per job. They don't need to know which node ran their job.

The network is coordinated through a control plane (a Next.js app backed by Supabase) that handles node registration, job routing, and payment accounting. Crucially, the control plane never holds private keys. All auth uses Nostr-style secp256k1 keypairs where nodes sign their own requests.

## How It Works at a High Level

1. **A node operator installs the CLI** and runs `infernet setup`. This generates a secp256k1 keypair, detects the GPU, installs an inference backend (Ollama by default), and registers the node on the network.
2. **The daemon starts** with `infernet start`. It heartbeats to the control plane every 30 seconds, reporting which models are loaded, GPU utilization, and availability.
3. **A client submits a job** via `POST /api/v1/jobs`. The control plane routes the job to a suitable node based on model availability, capacity, and proximity.
4. **The node runs inference** using whichever backend is installed (Ollama, vLLM, SGLang, MAX, or llama.cpp). Results stream back to the client via Server-Sent Events.
5. **Payment is settled.** The node's wallet receives a Compute Payment Receipt (CPR) that can be redeemed on-chain. Operators run `infernet payout` to claim earnings.

## What Makes It Different

**No vendor lock-in.** The API is consistent regardless of which backend is running inference. You can switch from Ollama to vLLM by changing an env var without changing your client code.

**Censorship resistance.** Because any operator can join and any model can be served, the network isn't dependent on any single company's content policies or business decisions.

**Real hardware.** Jobs run on real GPUs owned by real people, not virtual machines. Operators who invest in better hardware can handle larger models and charge more.

**Crypto-native payments.** Payments are settled on-chain with no intermediary holding funds. Operators get paid directly into their wallets.

**Open source.** The CLI, control plane code, and protocol spec are all open source. You can self-host the entire stack if you want full independence.

## What It Is Not

Infernet Protocol is not a model hosting service. It doesn't store model weights. Operators pull models onto their own machines and serve them from local storage.

It's not a training platform (yet — see Chapter 6 for the roadmap).

It's not a cloud provider. There are no SLAs, no guaranteed uptime per node, and no managed infrastructure. The network's reliability comes from having many nodes, not from any individual node being highly available.

## Chapter 2: Node Operators

Running a node is how you contribute compute to the network and earn crypto. This chapter covers everything from hardware selection through daily operations.

---

### In This Chapter

- **Requirements** — GPU tiers, RAM, bandwidth, and OS requirements. What hardware you can use and what to expect from each tier.
  - **Installation** — Full install walkthrough: the curl installer, `inferneth setup`, firewall rules, and running the daemon as a system service.
  - **Model Management** — Installing and removing models via the CLI and dashboard. How the daemon serves models and polls for updates.
  - **Monitoring** — `inferneth logs`, `inferneth status`, `inferneth doctor`, and what the dashboard shows you about your node's health.
  - **Earnings** — How payments work per job, how to check your balance, and how to run `inferneth payout` to claim earnings.
- 

### The Operator's Day-to-Day

Once your node is set up, operating it is mostly passive. The daemon runs in the background, accepts jobs automatically, and earns payments without your involvement. The main active tasks are:

- **Managing your model inventory:** adding new models when clients request them, removing models you're not getting jobs for to free VRAM.
- **Monitoring node health:** checking logs when something looks wrong, running `inferneth doctor` when the node goes offline unexpectedly.
- **Claiming earnings:** running `inferneth payout` periodically to move accumulated earnings to your wallet.
- **Keeping software current:** running `inferneth upgrade` when new CLI versions are released.

The rest of this chapter covers each of these in detail.

---

## Earnings

### How Payments Work

Every completed inference job generates a **Compute Payment Receipt (CPR)**. A CPR is a signed record that proves your node completed a specific job: it includes the job ID, token count, model used, your node's public key, the client's public key, and an on-chain settlement value.

CPRs accumulate in your node's account on the control plane. When you run `inferneth payout`, the pending CPRs are aggregated into an on-chain transaction that transfers the earned amount to your payout wallet.

## Checking Your Balance

```
infernet status
# Shows: Earnings: 12.84 USDC (unclaimed)
```

For more detail:

```
infernet payments
```

Earnings Summary

=====

Unclaimed balance: 12.84 USDC

Paid out (lifetime): 847.22 USDC

Recent jobs (last 24h):

|              |             |            |             |                  |
|--------------|-------------|------------|-------------|------------------|
| job_9a3f2c1d | qwen2.5:14b | 312 tokens | 0.0031 USDC | 2026-04-30 14:23 |
| job_8b2e1c3a | llama3.2:3b | 89 tokens  | 0.0009 USDC | 2026-04-30 14:19 |
| job_7c1d4b2f | qwen2.5:14b | 891 tokens | 0.0089 USDC | 2026-04-30 14:11 |
| ...          |             |            |             |                  |

Total today: 142 jobs, 0.9182 USDC

You can also see this in the dashboard under **Earnings** → **Ledger**.

## Setting Your Payout Address

Before you can receive payments, set your payout wallet address:

```
infernet payout --set-address 0xYourWalletAddressHere
```

Or edit ~/.infernet/config.json directly:

```
{
  "payout_address": "0xYourWalletAddressHere"
}
```

The address must be valid for the chain you want to receive payment on. Infernet supports multiple chains — specify which chain when setting the address:

*# Ethereum mainnet*

```
infernet payout --set-address 0x... --chain ethereum
```

*# Base*

```
infernet payout --set-address 0x... --chain base
```

*# Solana*

```
infernet payout --set-address YourSolanaPublicKey --chain solana
```

The address is sent to the control plane on the next heartbeat and appears in the dashboard under **Settings** → **Payout Address**.

## Claiming Earnings

```
infernet payout
```

This initiates an on-chain settlement for your current unclaimed balance. The process:

1. The CLI requests a payout from the control plane
2. The control plane aggregates all pending CPRs for your node
3. An on-chain transaction is submitted to transfer the balance to your payout address
4. The CLI waits for confirmation and shows the transaction hash

```
Initiating payout...
```

```
Unclaimed balance: 12.84 USDC
```

```
Destination: 0x1234...abcd
```

```
Chain: base
```

```
Submitting transaction...
```

```
Tx hash: 0xabc123...
```

```
Waiting for confirmation...
```

```
Confirmed! (block 14298741)
```

```
12.84 USDC transferred to 0x1234...abcd
```

Payout transactions typically confirm within 5–30 seconds on Base, a few minutes on Ethereum mainnet.

## Payout Minimums

There's a minimum payout threshold to avoid spending more on gas than you earn. The minimum varies by chain:

| Chain    | Minimum Payout |
|----------|----------------|
| Base     | 1.00 USDC      |
| Ethereum | 10.00 USDC     |
| Polygon  | 1.00 USDC      |
| Solana   | 1.00 USDC      |

If your balance is below the minimum, the payout command will tell you and exit without submitting a transaction.

## Pricing Per Job

Job pricing is set by the network based on:

- **Model:** larger models cost more per token
- **Token count:** output tokens + input tokens, weighted (output costs ~4x more)
- **Tier premium:** >=48gb tier nodes earn a small premium for serving models others can't

Current approximate rates (subject to network demand):

| Model Size | Rate (per 1K output tokens) |
|------------|-----------------------------|
| 1B–3B      | \$0.001                     |
| 7B         | \$0.002                     |
| 13B–14B    | \$0.005                     |
| 30B–33B    | \$0.012                     |
| 70B+       | \$0.025                     |

These rates are illustrative. The actual network rate is visible in the dashboard under **Earnings** → **Pricing**.

## Automated Payouts

You can schedule automatic payouts via cron. The `--auto` flag runs the payout non-interactively and only if the balance meets the minimum:

```
# Claim earnings every day at midnight
0 0 * * * /usr/local/bin/infernet payout --auto >> /var/log/infernet-payout.log 2>&1
```

Or configure auto-payout in `~/.infernet/config.json`:

```
{
  "auto_payout": true,
  "auto_payout_threshold": 5.0,
  "auto_payout_schedule": "daily"
}
```

Valid schedule values: "hourly", "daily", "weekly".

## Payment Security

Payments are cryptographically tied to your node's keypair. A CPR is only valid if it's signed by the node that executed the job. The control plane verifies the signature before issuing the on-chain transaction.

This means: even if someone gains access to your payout wallet address, they can't forge CPRs or claim earnings that aren't yours. And even if someone gains access to the control plane, they can't redirect your earnings because the on-chain transaction requires a valid CPR signature.

Keep your node's private key (`~/.infernet/keys/node.key`) secure. Back it up somewhere safe. If you lose it, you can't claim any pending unclaimed earnings, and you'll need to re-register with a new keypair.

## Installation

### Install the CLI

```
curl -sSL https://infernet.sh/install | bash
```

The installer: 1. Detects your OS and architecture 2. Downloads the appropriate binary 3. Installs it to `~/.local/bin` (or `/usr/local/bin` with `sudo`) 4. Adds it to your `PATH` in `~/.bashrc` / `~/.zshrc`

After the install, either open a new shell or run:

```
source ~/.bashrc  
# or  
source ~/.zshrc
```

Verify:

```
infernet --version
```

## Manual Install

If you prefer not to pipe to bash:

```
# Check latest version  
INFERNET_VERSION=$(curl -s https://infernet.sh/version)  
  
# Download binary for your platform  
curl -sSL "https://github.com/infernetprotocol/infernet/releases/download/${INFERNET_VERSION}/infernet-${INFERNET_VERSION}.tar.gz" -o /usr/local/bin/infernet  
  
chmod +x /usr/local/bin/infernet
```

Available platforms: linux-x86\_64, linux-aarch64, darwin-arm64, darwin-x86\_64.

---

## Run Setup

```
infernet setup
```

### What Setup Does

#### 1. Collect registration token

You'll be prompted for the one-time registration token from the dashboard. Get it at **Nodes** → **Add Node** in the Infernet Dashboard.

#### 2. Detect hardware

The wizard runs `nvidia-smi`, `rocm-smi`, and system checks to determine: - GPU model and VRAM - VRAM tier assignment - System RAM - Number of CPU cores

#### 3. Install inference backend

If no backend is detected, Ollama is installed automatically. If you want a different backend, install it first (see Chapter 3), then run `setup` — it will detect the backend and use it.

#### 4. Pull default model

The wizard suggests a default model based on your tier:



| Tier   | Default Model |
|--------|---------------|
| >=48gb | qwen2.5:72b   |
| >=24gb | qwen2.5:14b   |
| >=12gb | qwen2.5:7b    |
| >=8gb  | qwen2.5:7b    |
| cpu    | qwen2.5:1.5b  |

You can accept the default or enter a different model name. The wizard pulls the model before proceeding.

## 5. Generate keypair

A secp256k1 keypair is generated for this node. The private key is written to `~/.infernnet/keys/node.key` with permissions 600. The public key is registered with the control plane.

## 6. Write config

Config is written to `~/.infernnet/config.json`:

```
{
  "node_id": "node_8f3a2c1d",
  "public_key": "npub1abc123...",
  "control_plane_url": "https://app.infernnet.sh",
  "backend": "ollama",
  "ollama_host": "http://localhost:11434",
  "served_models": ["qwen2.5:14b"],
  "vram_tier": ">=24gb",
  "payout_address": ""
}
```

You can edit this file directly. Changes take effect on next daemon restart.

---

## Firewall Configuration

The node daemon listens on port 3000 by default. The inference backend (e.g., Ollama) listens on its own port (11434 for Ollama). Neither needs to be externally accessible — the daemon makes outbound connections to the control plane only.

However, if you're running behind a NAT or firewall, make sure outbound HTTPS (port 443) is permitted:

```
# UFW (Ubuntu)
sudo ufw allow out 443/tcp
sudo ufw allow out 80/tcp

# Check current status
sudo ufw status
```

If clients connect directly to your node (direct-routing mode), you also need inbound on port 3000:

```
sudo ufw allow in 3000/tcp
```

For datacenter deployments with iptables:

```
# Allow outbound HTTPS
iptables -A OUTPUT -p tcp --dport 443 -j ACCEPT

# Allow inbound on node port (if using direct routing)
iptables -A INPUT -p tcp --dport 3000 -j ACCEPT
```

---

## Running as a System Service

For production nodes, you want the daemon to start automatically on boot and restart if it crashes.

```
inferneta service install
```

This creates a systemd service unit at `/etc/systemd/system/inferneta.service` (Linux) or a launchd plist at `~/Library/LaunchAgents/sh.inferneta.daemon.plist` (macOS).

## Managing the Service

```
# Start
inferneta service start

# Stop
inferneta service stop

# Restart
inferneta service restart

# Check status
inferneta service status

# View logs (follows systemd journal)
inferneta service logs

# Remove from boot (does not uninstall)
inferneta service uninstall
```

## Manual Systemd Unit

If you prefer to write your own unit file:

```
[Unit]
Description=Infernet Protocol Node Daemon
After=network.target
Wants=network.target
```

```
[Service]
Type=simple
User=infernet
ExecStart=/usr/local/bin/infernet start
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal
Environment=HOME=/home/infernet
```

```
[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable infernet
sudo systemctl start infernet
```

It's good practice to run the daemon as a dedicated non-root user. Create one with:

```
sudo useradd -r -s /bin/false -d /home/infernet infernet
sudo mkdir -p /home/infernet/.infernet
sudo chown -R infernet:infernet /home/infernet
```

Then move your config:

```
sudo cp -r ~/.infernet/* /home/infernet/.infernet/
sudo chown -R infernet:infernet /home/infernet/.infernet
```

---

## Upgrading the CLI

```
infernet upgrade
```

This re-runs the curl installer to fetch the latest version. Your config and keys are preserved.

---

## Uninstalling

```
infernet remove
```

This stops the daemon, removes the service unit, removes the binary, and optionally removes your config and keys (it will ask). The node's registration on the control plane is deactivated.

---

## Model Management

Models are the inventory of your node. The more relevant models you have loaded, the more jobs you'll match. Managing this inventory well is one of the primary levers you have over your node's earnings.

## How Served Models Work

Your node's config has a `served_models` array. This is the canonical list of models your node advertises to the network:

```
{
  "served_models": ["qwen2.5:14b", "llama3.2:3b", "deepseek-coder:6.7b"]
}
```

When the daemon heartbeats, it reports these models to the control plane. The job router uses this list to determine which jobs your node can accept. A job for `qwen2.5:14b` will only be routed to nodes that list it in `served_models`.

The models must actually be loaded in the inference backend. `served_models` and what's actually pulled in your backend (e.g., `ollama list`) should stay in sync. The daemon checks this at startup and warns if there's a mismatch.

## Installing Models via CLI

```
infernet model install qwen2.5:14b
```

This does three things: 1. Pulls the model into the inference backend (`ollama pull qwen2.5:14b` for Ollama backends) 2. Adds the model to `served_models` in your config 3. Sends a sync heartbeat so the control plane updates immediately (no need to wait 30 seconds)

Watch the pull progress:

```
infernet model install qwen2.5:14b --verbose  
# Pulling qwen2.5:14b from Ollama registry...  
# ██████████ 89% (7.2 GB / 8.1 GB) 45 MB/s
```

For vLLM or SGLang backends, specify the HuggingFace model ID:

```
infernet model install Qwen/Qwen2.5-14B-Instruct
```

## Removing Models via CLI

```
infernet model remove qwen2.5:14b
```

This: 1. Removes the model from served\_models 2. Deletes the weights from the backend 3. Syncs with the control plane

The node will stop receiving jobs for this model immediately.

## Listing Installed Models

```
infernet model list
```

| Model               | Size   | Backend | Status |
|---------------------|--------|---------|--------|
| qwen2.5:14b         | 8.1 GB | ollama  | loaded |
| llama3.2:3b         | 2.0 GB | ollama  | loaded |
| deepseek-coder:6.7b | 3.8 GB | ollama  | loaded |

## Installing Models via Dashboard

In the Infernet Dashboard, navigate to your node's detail page and click **Manage Models**. You'll see:

- **Installed models:** currently in your `served_models`
- **Suggested models:** models with active job demand that your tier can run
- **Browse:** search the full model catalog

Click **Install** on any model. The command is queued in the control plane's command queue and picked up by the daemon on its next heartbeat cycle (within 30 seconds).

The daemon polls the command queue on every heartbeat tick. Commands are structured like:

```
{
  "type": "model_install",
  "model": "qwen2.5:32b",
  "issued_at": "2026-04-30T14:00:00Z"
}
```

The daemon executes the install, updates `served_models`, and acknowledges the command back to the control plane.

## Model Naming Conventions

Models are identified by the format used by the installed backend:

### Ollama:

qwen2.5:7b  
qwen2.5:14b  
qwen2.5:72b  
llama3.2:3b  
llama3.1:8b  
deepseek-coder:6.7b  
nomic-embed-text:latest

### vLLM / SGLang / MAX (HuggingFace repo IDs):

Qwen/Qwen2.5-7B-Instruct  
Qwen/Qwen2.5-14B-Instruct  
meta-llama/Llama-3.1-8B-Instruct  
deepseek-ai/DeepSeek-V2-Lite

### llama.cpp (GGUF filenames):

qwen2.5-7b-instruct-q4\_k\_m.gguf  
llama-3.1-8b-instruct-q5\_k\_m.gguf

When installing via the CLI, use the naming format appropriate for your backend. The control plane normalizes model names for routing.

## VRAM Planning

Before installing a model, check that it fits in your VRAM. Rule of thumb: a model needs roughly 2 bytes per parameter for FP16, 1 byte per parameter for INT8, and 0.5 bytes per parameter for Q4.

| Model | FP16 VRAM | INT8 VRAM | Q4 VRAM |
|-------|-----------|-----------|---------|
| 3B    | 6 GB      | 3 GB      | 1.5 GB  |
| 7B    | 14 GB     | 7 GB      | 3.5 GB  |
| 14B   | 28 GB     | 14 GB     | 7 GB    |
| 32B   | 64 GB     | 32 GB     | 16 GB   |
| 70B   | 140 GB    | 70 GB     | 35 GB   |

With Ollama, quantized variants are pulled by default. Specify the tag explicitly to get a particular quantization:

```
# Q4_K_M quantized (smaller, faster)
infernet model install qwen2.5:14b-instruct-q4_K_M

# FP16 (full precision, needs full VRAM)
infernet model install qwen2.5:14b-instruct-fp16
```

## Hot-Swap vs Cold-Load

Ollama keeps loaded models in VRAM until memory pressure requires eviction. If you have multiple models installed, Ollama will swap them in and out as needed. This means:

- First inference on a model after a swap takes 5–30 seconds (load time)
- Subsequent inferences are fast

If you want a model to always be hot (no load latency), limit your `served_models` to the number of models that fit simultaneously in your VRAM, and tell Ollama to keep them loaded:

```
# Keep model in VRAM indefinitely (Ollama-specific)
OLLAMA_KEEP_ALIVE=-1 ollama run qwen2.5:14b
```

Or set the env var in your config and restart the daemon.

---

## Monitoring Your Node

### Status Overview

```
infernet status
```

```
Node:          node_8f3a2c1d
Status:        online
Uptime:        3d 14h 22m
Backend:       ollama (localhost:11434)
Models loaded: qwen2.5:14b, llama3.2:3b
```

```
GPU:           NVIDIA RTX 4090 (24 GB) – 61% utilized
Jobs today:    142 completed, 0 failed, 1 pending
Earnings:      12.84 USDC (unclaimed)
Last heartbeat: 18 seconds ago
```

Run it with `--json` to get machine-readable output:

```
inferneter status --json | jq '.gpu.utilization'
```

## Logs

### Following Live Logs

```
inferneter logs
```

Tails the daemon log in real time. Press Ctrl+C to stop.

Example output:

```
[2026-04-30 14:23:32] Heartbeat OK (latency: 39ms)
[2026-04-30 14:23:41] Job job_9a3f2c1d received: qwen2.5:14b, 512 ctx
[2026-04-30 14:23:41] Routing to backend: ollama
[2026-04-30 14:23:42] Streaming started (job_9a3f2c1d)
[2026-04-30 14:23:48] Job job_9a3f2c1d complete: 312 tokens, 52 tok/s
[2026-04-30 14:23:48] CPR issued: cpr_7x2a1b3c
[2026-04-30 14:24:02] Heartbeat OK (latency: 41ms)
```

### Log History

```
inferneter logs --lines 500
inferneter logs --since 1h
inferneter logs --since "2026-04-30 12:00"
```

### Log Levels

```
inferneter logs --level debug
```

Debug logs include: incoming request details, auth header verification, backend probe results, and command queue polling.

### Log Files

Log files are written to `~/.inferneter/logs/`:

```
~/.inferneter/logs/
daemon.log           # Main daemon log (current)
daemon.log.1         # Rotated log
daemon.log.2
backend.log          # Inference backend output
install.log          # Setup and model install log
```

Logs rotate at 50 MB. Up to 5 rotated files are kept.

## Doctor

infern timer doctor runs a diagnostic suite and reports any issues:

```
infern timer doctor
```

```
Infer timer Node Diagnostics
```

```
=====
```

```
[OK] CLI version: 0.9.2 (latest)
[OK] Config file: ~/.infern timer/config.json
[OK] Node keypair: present
[OK] Control plane: reachable (42ms)
[OK] Node registered: node_8f3a2c1d
[OK] Node online: yes (last heartbeat 12s ago)
[OK] Backend (ollama): running at localhost:11434
[OK] Models: qwen2.5:14b (loaded), llama3.2:3b (loaded)
[WARN] served_models in config: 2 models
       served_models in backend: 2 models (match)
[OK] Firewall: outbound 443 accessible
[OK] Disk space: 234 GB free
[OK] GPU drivers: NVIDIA 545.29.06
[OK] CUDA: 12.4
[OK] GPU memory: 24 GB (7.2 GB used, 16.8 GB free)
```

All checks passed (1 warning).

Common issues doctor catches:

| Issue               | Symptom             | Doctor Output                                              |
|---------------------|---------------------|------------------------------------------------------------|
| Backend not running | Node shows offline  | [FAIL] Backend (ollama): not reachable at localhost:11434  |
| Model mismatch      | Jobs fail silently  | [WARN] served_models mismatch: config has X, backend has Y |
| Clock skew          | Auth failures       | [FAIL] System time: 4m 12s drift from NTP (max 30s)        |
| Disk full           | Model installs fail | [FAIL] Disk space: 2 GB free (minimum 20 GB recommended)   |
| Outdated CLI        | Missing features    | [WARN] CLI version: 0.8.1 (latest is 0.9.2)                |

Run doctor first whenever your node is behaving unexpectedly.

## Dashboard Monitoring

The Infer timer Dashboard shows your node's status in real time via Supabase Realtime:

- **Status dot:** green (online), yellow (degraded), red (offline)



- **Heartbeat:** time since last successful heartbeat
- **Version:** CLI version reported on last heartbeat — useful for knowing when you need to upgrade
- **Models:** which models are currently loaded
- **Job stats:** jobs completed today, this week, all time
- **Earnings:** unclaimed USDC balance
- **GPU metrics:** utilization and memory usage, updated every heartbeat

The **Recent Jobs (Your Nodes Processed)** panel shows the last 20 jobs your node handled, with timestamps, model used, token count, latency, and payment received.

## Heartbeat Intervals

The daemon heartbeats every **30 seconds**. If 3 consecutive heartbeats fail (90 seconds), the control plane marks the node offline. Jobs stop being routed to it immediately.

When the node comes back online and the next heartbeat succeeds, it's marked online again and job routing resumes.

You can check the last heartbeat time from the CLI:

```
infernet status | grep "Last heartbeat"
```

Or query it directly:

```
curl -s https://app.infernet.sh/api/v1/nodes/node_8f3a2c1d \
-H "Authorization: Bearer $INFERNET_BEARER_TOKEN" | jq '.last_heartbeat'
```

## Alerting

For production nodes, consider setting up uptime monitoring on your node's heartbeat. The control plane exposes a public status endpoint:

GET https://app.infernet.sh/api/v1/nodes/{node\_id}/status

Returns {"status": "online"} or {"status": "offline"} — easy to wire into any uptime monitoring tool (UptimeRobot, Healthchecks.io, etc.).

You can also configure webhook notifications in the dashboard: **Settings** → **Notifications** to get a Slack or Discord ping when your node goes offline.

---

## Hardware Requirements

### GPU Tiers

Infernet classifies nodes into VRAM tiers. Your tier determines which models your node can serve and how you're matched to jobs. Higher tiers get more job volume and can charge more per job.

**Tier: >=48gb (Flagship)**

**Example hardware:** NVIDIA H100 80GB, A100 80GB, RTX 6000 Ada (48GB)

**What you can run:** 70B parameter models (Qwen2.5-72B, Llama-3.1-70B), large code models (DeepSeek Coder 33B), multi-modal models with large context windows.

**Expected throughput:** 50–150 tokens/second for 70B models. 200+ tokens/second for 7B–13B models.

**Recommended backends:** vLLM or SGLang for maximum throughput. Both support tensor parallelism if you have multiple GPUs.

**Tier: >=24gb (High-End Consumer / Professional)**

**Example hardware:** NVIDIA RTX 4090 (24GB), RTX 3090 (24GB), A5000 (24GB)

**What you can run:** 14B–33B models at full precision, 70B models with aggressive quantization (Q4).

**Expected throughput:** 30–80 tokens/second for 14B models, 15–30 tokens/second for 33B.

**Recommended backends:** Ollama works well. vLLM for higher throughput if you’re getting steady job volume.

**Tier: >=12gb (Mid-Range)**

**Example hardware:** NVIDIA RTX 4080 (16GB), RTX 3080 (12GB), A2000 (12GB)

**What you can run:** 7B–13B models at full precision, 14B–30B with Q4 quantization.

**Expected throughput:** 40–90 tokens/second for 7B models.

**Recommended backends:** Ollama. llama.cpp if you need GGUF model support.

**Tier: >=8gb (Entry)**

**Example hardware:** NVIDIA RTX 4060 Ti (8GB), RTX 3070 (8GB), AMD RX 6800 (16GB)

**What you can run:** 7B models at full precision, larger models with heavy quantization.

**Expected throughput:** 20–50 tokens/second for 7B models depending on memory bandwidth.

**Notes:** AMD GPUs in this tier work well with Ollama via ROCm. Apple Silicon (M1 Pro, M2) fits here too, with Ollama or llama.cpp.

**Tier: cpu (CPU-Only)**

**What you can run:** Small models (1B–3B), heavily quantized 7B models. Not recommended for production job-taking — CPU inference is very slow for most clients.

**Best use:** Development, testing your setup, serving small specialized models (embedding models, classifiers).

---

**RAM**

| GPU Tier | Minimum RAM | Recommended RAM |
|----------|-------------|-----------------|
| >=48gb   | 64 GB       | 128 GB          |

| GPU Tier | Minimum RAM | Recommended RAM |
|----------|-------------|-----------------|
| >=24gb   | 32 GB       | 64 GB           |
| >=12gb   | 16 GB       | 32 GB           |
| >=8gb    | 16 GB       | 32 GB           |
| cpu      | 16 GB       | 32 GB           |

vLLM and SGLang manage memory aggressively and benefit from having more system RAM available for KV-cache spill. Ollama is less demanding.

## Storage

Model weights take significant disk space. Budget accordingly:

| Model Size | Approximate Disk (FP16) | Approximate Disk (Q4) |
|------------|-------------------------|-----------------------|
| 1B–3B      | 2–6 GB                  | 0.5–2 GB              |
| 7B         | 14 GB                   | 4 GB                  |
| 13B–14B    | 28 GB                   | 8 GB                  |
| 30B–33B    | 65 GB                   | 18 GB                 |
| 70B–72B    | 140 GB                  | 40 GB                 |

Use fast storage (NVMe SSD) for model weights. Model loading time on cold start is much faster with NVMe vs HDD or SATA SSD.

Minimum recommendations: - **>=48gb tier:** 1 TB NVMe - **>=24gb tier:** 500 GB NVMe - **<=12gb tier:** 250 GB NVMe

## Bandwidth

Minimum: **100 Mbps** symmetric.

The bandwidth bottleneck is model downloads (one-time) and inference results (streaming). A typical streaming inference response is a few KB/second per active job. At 10 concurrent jobs, you’re looking at 50–100 KB/s of upstream.

More important than raw bandwidth is **upload latency**. High-latency connections (>100ms) degrade the streaming experience for clients. Datacenter connections are strongly preferred over residential for >=24gb tier and above.

## Operating System

**Strongly recommended:** Ubuntu 22.04 LTS or Ubuntu 24.04 LTS.

The installer and daemon are tested primarily on Ubuntu. Most of the tooling in the ecosystem (NVIDIA drivers, CUDA, ROCm, Docker) has the best support on Ubuntu.

**Supported:** Debian 11+, other Debian-based distros, Fedora 38+, CentOS Stream 9.

**macOS:** Supported for development and Apple Silicon nodes. `infernet setup` works on macOS. llama.cpp and Ollama both support Metal. Production deployment on macOS is reasonable for smaller nodes.

**Windows:** Not currently supported. Use WSL2 if you need to test on Windows.

---

## GPU Drivers

Install GPU drivers **before** running `infernet setup`. The setup wizard detects your GPU via `nvidia-smi` (NVIDIA) or `rocm-smi` (AMD) and will warn you if drivers are missing.

**NVIDIA:** Install the latest stable driver from [developer.nvidia.com](https://developer.nvidia.com). CUDA 12.1+ required for vLLM and SGLang. Ollama manages its own CUDA version.

**AMD:** ROCm 5.7+ for RX 6000/7000 series. Install via the official ROCm installer. Ollama has ROCm support built in.

**Apple Silicon:** No additional drivers needed. Metal is used automatically.

Quick driver check:

```
# NVIDIA
nvidia-smi

# AMD
rocm-smi

# Apple Silicon
system_profiler SPDisplaysDataType | grep "Metal"
```

---

## Chapter 3: Inference Backends

Infernet Protocol supports five inference backends. The daemon auto-detects which one to use, but understanding your options lets you pick the right tool for your hardware and workload.

---

### In This Chapter

- Ollama — The default. Easy setup, broad hardware support (NVIDIA, AMD, Apple Silicon, CPU).
- vLLM — High throughput on NVIDIA hardware. Best for nodes with heavy job volume.
- SGLang — KV-cache reuse and structured output. Best when clients use similar prompt prefixes.
- Modular MAX — High throughput with a different memory management approach. Competitive with vLLM.

- llama.cpp — No Python dependency. CPU and Apple Silicon native. GGUF model format.
  - Choosing a Backend — Decision guide with a comparison table.
- 

## How Auto-Selection Works

At startup, the daemon probes each backend in priority order by sending a health check request. The first backend that responds is used.

Default probe order: 1. Ollama (localhost:11434/api/tags) 2. vLLM (localhost:8000/health) 3. SGLang (localhost:30000/health) 4. Modular MAX (localhost:8080/health) 5. llama.cpp / llama-swap (localhost:8080/health)

Override the selection with an env var:

```
export INFERNET_BACKEND=vllm
infernet start
```

Or in your config:

```
{
  "backend": "vllm"
}
```

---

## Backend Adapter Interface

All backends speak to the daemon through a common adapter interface:

- POST /generate — single inference request
- POST /generate/stream — streaming inference request
- GET /models — list loaded models
- GET /health — health check

The daemon translates between this interface and each backend's native API. You don't need to know the backend's native API unless you're doing advanced configuration.

---